tpcc_dbfs_vs_postgres_basic_comparison

May 4, 2026

1 TPCC Comparison: dbfs vs PostgreSQL

This notebook compares two BenchBase TPCC runs collected under the same nominal settings and grouped under a shared comparison label:

- comparison label: `20260503`
- artifact selection: latest BenchBase artifact prefix under each engine directory
- dbfs result directory: `benchmarking/results/20260503/tpcc/dbfs`
- PostgreSQL result directory: `benchmarking/results/20260503/tpcc/postgres`

The notebook reads summary, metrics, params, samples, results, raw traces, per-procedure result windows, and optional query-trace CSVs from those directories to build side-by-side comparison tables and performance plots.

[1]:
```
from pathlib import Path
import json
import re

import matplotlib.pyplot as plt
import pandas as pd
from IPython.display import display

plt.style.use('ggplot')
pd.set_option('display.float_format', lambda value: f'{value:,.3f}')

comparison_root = Path(
    '/Users/yohasebe/Documents/dbfs/benchmarking/results/20260503/tpcc')

run_configs = {
    'dbfs': {
        'label': 'tpcc_2026-05-03_04-12-31',
        'display_name': 'dbfs',
        'result_dir': comparison_root / 'dbfs',
    },
    'postgres': {
        'label': 'tpcc_2026-05-03_04-14-31',
        'display_name': 'PostgreSQL',
        'result_dir': comparison_root / 'postgres',
    },
```

```
}

configured_ratios = pd.Series(
    {
        'NewOrder': 45.0,
        'Payment': 43.0,
        'OrderStatus': 4.0,
        'Delivery': 4.0,
        'StockLevel': 4.0,
    },
    name='Configured Ratio (%)')

required_suffixes = {
    'summary': '.summary.json',
    'metrics': '.metrics.json',
    'params': '.params.json',
    'results': '.results.csv',
    'samples': '.samples.csv',
    'raw': '.raw.csv',
}

query_trace_filenames = {
    'dbfs': 'dbfs_query_trace.csv',
    'postgres': 'postgres_query_trace.csv',
}
procedure_suffix = '.results.'
```

[2]:
```
dbfs_terminal_log = '''
[INFO ] 2026-05-03 04:09:32,266 [main]  com.oltpbenchmark.DBWorkload main -␣
 ↪========================================================================
Benchmark:                          TPCC {com.oltpbenchmark.benchmarks.tpcc.
 ↪TPCCBenchmark}
Configuration:                      /benchbase/benchbase-config/dbfs_tpcc_docker.
 ↪xml
Type:                               POSTGRES
Driver:                             dev.yohaku.dbfs.jdbc.DbfsDriver
URL:                                jdbc:dbfs://dbfs:25432/benchbase
Isolation:                          TRANSACTION_SERIALIZABLE
Batch Size:                         128
DDL Path:                           null
Loader Threads:                     2
Session Setup File:                 null
Scale Factor:                       1.0
Terminals:                          1
New Connection Per Txn:             false
Reconnect on Connection Failure: true
```

```
[INFO ] 2026-05-03 04:11:29,210 [main]  com.oltpbenchmark.ThreadBench␣
 ↪runRateLimitedMultiPhase - PHASE START :: [Workload=TPCC] [Serial=false]␣
 ↪[Time=60] [WarmupTime=0] [Rate=10000.0] [Arrival=REGULAR] [Ratios=[45.0, 43.
 ↪0, 4.0, 4.0, 4.0]] [ActiveWorkers=1]
[INFO ] 2026-05-03 04:12:30,875 [main]  com.oltpbenchmark.DBWorkload␣
 ↪runWorkload - Rate limited reqs/s: Results(state=EXIT,␣
 ↪nanoSeconds=61382252527, measuredRequests=324) = 5.278398668368895 requests/
 ↪sec (throughput), 5.310981376198332 requests/sec (goodput)
Completed Transactions:
com.oltpbenchmark.benchmarks.tpcc.procedures.NewOrder/01                    ␣
 ↪ [ 145]␣
 ↪**************************************************************************
com.oltpbenchmark.benchmarks.tpcc.procedures.Payment/02                     ␣
 ↪ [ 136]␣
 ↪**************************************************************************
com.oltpbenchmark.benchmarks.tpcc.procedures.OrderStatus/03                 ␣
 ↪ [  18] *********
com.oltpbenchmark.benchmarks.tpcc.procedures.Delivery/04                    ␣
 ↪ [  14] *******
com.oltpbenchmark.benchmarks.tpcc.procedures.StockLevel/05                  ␣
 ↪ [  13] ******
Aborted Transactions:
<EMPTY>
Unexpected SQL Errors:
<EMPTY>
'''

postgres_terminal_log = '''
[INFO ] 2026-05-03 04:13:02,786 [main]  com.oltpbenchmark.DBWorkload main -␣
 ↪=========================================================================
Benchmark:                          TPCC {com.oltpbenchmark.benchmarks.tpcc.
 ↪TPCCBenchmark}
Configuration:                      /benchbase/benchbase-config/
 ↪postgres_tpcc_docker.xml
Type:                               POSTGRES
Driver:                             org.postgresql.Driver
URL:                                jdbc:postgresql://postgres:5432/benchbase?
 ↪sslmode=disable&ApplicationName=tpcc&reWriteBatchedInserts=true
Isolation:                          TRANSACTION_SERIALIZABLE
Batch Size:                         128
DDL Path:                           null
Loader Threads:                     2
Session Setup File:                 null
Scale Factor:                       1.0
Terminals:                          1
New Connection Per Txn:             false
```

```
Reconnect on Connection Failure: true
[INFO ] 2026-05-03 04:13:30,544 [main]  com.oltpbenchmark.ThreadBench␣
 ↪runRateLimitedMultiPhase - PHASE START :: [Workload=TPCC] [Serial=false]␣
 ↪[Time=60] [WarmupTime=0] [Rate=10000.0] [Arrival=REGULAR] [Ratios=[45.0, 43.
 ↪0, 4.0, 4.0, 4.0]] [ActiveWorkers=1]
[INFO ] 2026-05-03 04:14:30,824 [main]  com.oltpbenchmark.DBWorkload␣
 ↪runWorkload - Rate limited reqs/s: Results(state=EXIT,␣
 ↪nanoSeconds=60000046819, measuredRequests=24446) = 407.43301540656086␣
 ↪requests/sec (throughput), 405.2163504695948 requests/sec (goodput)
Completed Transactions:
com.oltpbenchmark.benchmarks.tpcc.procedures.NewOrder/01                    ␣
 ↪ [11103]␣
 ↪*****************************************************************************
com.oltpbenchmark.benchmarks.tpcc.procedures.Payment/02                     ␣
 ↪ [10298]␣
 ↪*****************************************************************************
com.oltpbenchmark.benchmarks.tpcc.procedures.OrderStatus/03                 ␣
 ↪ [  959] ******
com.oltpbenchmark.benchmarks.tpcc.procedures.Delivery/04                    ␣
 ↪ [ 1007] *******
com.oltpbenchmark.benchmarks.tpcc.procedures.StockLevel/05                  ␣
 ↪ [  946] ******
Aborted Transactions:
com.oltpbenchmark.benchmarks.tpcc.procedures.NewOrder/01                    ␣
 ↪ [ 134]␣
 ↪*****************************************************************************
Unexpected SQL Errors:
<EMPTY>
'''

run_configs['dbfs']['terminal_log'] = dbfs_terminal_log
run_configs['postgres']['terminal_log'] = postgres_terminal_log

field_patterns = {
    'workload': re.compile(r'Benchmark:\s+(\w+)'),
    'dbms_type': re.compile(r'Type:\s+(\w+)'),
    'driver': re.compile(r'Driver:\s+(.+)'),
    'url': re.compile(r'URL:\s+(.+)'),
    'isolation': re.compile(r'Isolation:\s+(\S+)'),
    'loader_threads': re.compile(r'Loader Threads:\s+(\d+)'),
    'scale_factor': re.compile(r'Scale Factor:\s+([\d.]+)'),
    'terminals': re.compile(r'Terminals:\s+(\d+)'),
}
phase_pattern = re.compile(
    r'PHASE START :: .*\[Time=(\d+)\] \[WarmupTime=(\d+)\] \[Rate=([\d.]+)\].
 ↪*\[Ratios=\[([^\]]+)\]\]'
```

```
)
rate_pattern = re.compile(
    r'measuredRequests=(\d+)\) = ([\d.]+) requests/sec \(throughput\), ([\d.]+)␣
 ↪requests/sec \(goodput\)'
)


def parse_terminal_log(log_text):
    parsed = {}
    for key, pattern in field_patterns.items():
        match = pattern.search(log_text)
        parsed[key] = match.group(1).strip() if match else None

    phase_match = phase_pattern.search(log_text)
    if phase_match:
        parsed['phase_time_seconds'] = int(phase_match.group(1))
        parsed['warmup_time_seconds'] = int(phase_match.group(2))
        parsed['target_rate'] = float(phase_match.group(3))
        parsed['configured_ratio_list'] = [
            float(value.strip()) for value in phase_match.group(4).split(',')
        ]

    rate_match = rate_pattern.search(log_text)
    if rate_match:
        parsed['terminal_measured_requests'] = int(rate_match.group(1))
        parsed['terminal_throughput'] = float(rate_match.group(2))
        parsed['terminal_goodput'] = float(rate_match.group(3))

    return parsed


parsed_configs = pd.DataFrame([{
    'engine': engine,
    **parse_terminal_log(config['terminal_log'])
} for engine, config in run_configs.items()])
parsed_configs
```

[2]:

```
     engine workload dbms_type                          driver  \
0      dbfs     TPCC  POSTGRES  dev.yohaku.dbfs.jdbc.DbfsDriver
1  postgres     TPCC  POSTGRES            org.postgresql.Driver

                                          url  \
0                  jdbc:dbfs://dbfs:25432/benchbase
1  jdbc:postgresql://postgres:5432/benchbase?sslm…

                  isolation loader_threads scale_factor terminals  \
0  TRANSACTION_SERIALIZABLE              2          1.0         1
```

```
1  TRANSACTION_SERIALIZABLE              2          1.0         1

   phase_time_seconds  warmup_time_seconds  target_rate  \
0                  60                    0   10,000.000
1                  60                    0   10,000.000

         configured_ratio_list  terminal_measured_requests  \
0  [45.0, 43.0, 4.0, 4.0, 4.0]                          324
1  [45.0, 43.0, 4.0, 4.0, 4.0]                        24446

   terminal_throughput  terminal_goodput
0                5.278             5.311
1              407.433           405.216
```

[3]:
```python
def find_latest_artifact_prefix(result_dir):
    summary_suffix = required_suffixes['summary']
    matches = sorted(result_dir.glob(f'*{summary_suffix}'))
    if not matches:
        raise FileNotFoundError(
            f'No artifact ending with {summary_suffix} under {result_dir}')
    latest_summary = matches[-1]
    return latest_summary.name[:-len(summary_suffix)]


def find_artifact(result_dir, run_label, suffix):
    artifact_path = result_dir / f'{run_label}{suffix}'
    if not artifact_path.exists():
        raise FileNotFoundError(
            f'No artifact ending with {suffix} for {run_label} under␣
 ↪{result_dir}'
        )
    return artifact_path


def load_optional_query_trace(engine, result_dir):
    trace_path = result_dir / query_trace_filenames[engine]
    if not trace_path.exists():
        return trace_path, None
    return trace_path, pd.read_csv(trace_path)


def load_run_artifacts(engine, config):
    result_dir = config['result_dir']
    detected_label = find_latest_artifact_prefix(result_dir)
    artifacts = {
        name: find_artifact(result_dir, detected_label, suffix)
        for name, suffix in required_suffixes.items()
```

```
    }
    procedure_paths = sorted(
        result_dir.glob(f'{detected_label}{procedure_suffix}*.csv'))
    query_trace_path, query_trace_df = load_optional_query_trace(
        engine, result_dir)

    with artifacts['summary'].open() as handle:
        summary = json.load(handle)
    with artifacts['metrics'].open() as handle:
        metrics = json.load(handle)
    with artifacts['params'].open() as handle:
        params = json.load(handle)

    results_df = pd.read_csv(artifacts['results'])
    samples_df = pd.read_csv(artifacts['samples'])
    raw_df = pd.read_csv(artifacts['raw'])
    procedure_dfs = {
        path.stem.split('.results.', 1)[1]: pd.read_csv(path)
        for path in procedure_paths
    }

    return {
        'label': detected_label,
        'artifacts': artifacts,
        'summary': summary,
        'metrics': metrics,
        'params': params,
        'results_df': results_df,
        'samples_df': samples_df,
        'raw_df': raw_df,
        'procedure_dfs': procedure_dfs,
        'query_trace_path': query_trace_path,
        'query_trace_df': query_trace_df,
    }


runs = {}
validation_rows = []
for engine, config in run_configs.items():
    run_data = load_run_artifacts(engine, config)
    runs[engine] = {**config, **run_data}
    validation_rows.extend([
        {
            'engine': engine,
            'artifact': 'results.csv',
            'rows': len(run_data['results_df']),
            'columns': ', '.join(run_data['results_df'].columns),
```

```
        },
        {
            'engine': engine,
            'artifact': 'samples.csv',
            'rows': len(run_data['samples_df']),
            'columns': ', '.join(run_data['samples_df'].columns),
        },
        {
            'engine': engine,
            'artifact': 'raw.csv',
            'rows': len(run_data['raw_df']),
            'columns': ', '.join(run_data['raw_df'].columns),
        },
        {
            'engine': engine,
            'artifact': 'procedure_results',
            'rows': len(run_data['procedure_dfs']),
            'columns': ', '.join(sorted(run_data['procedure_dfs'].keys())),
        },
        {
            'engine':
            engine,
            'artifact':
            query_trace_filenames[engine],
            'rows':
            0 if run_data['query_trace_df'] is None else len(
                run_data['query_trace_df']),
            'columns':
            'missing (optional)' if run_data['query_trace_df'] is None else
            ', '.join(run_data['query_trace_df'].columns),
        },
    ])

validation_df = pd.DataFrame(validation_rows)
validation_df
```

[3]:

	engine	artifact	rows	\
0	dbfs	results.csv	12	
1	dbfs	samples.csv	60	
2	dbfs	raw.csv	324	
3	dbfs	procedure_results	5	
4	dbfs	dbfs_query_trace.csv	69	
5	postgres	results.csv	12	
6	postgres	samples.csv	60	
7	postgres	raw.csv	24858	
8	postgres	procedure_results	5	
9	postgres	postgres_query_trace.csv	69	

```
                                                columns
0  Time (seconds), Throughput (requests/second), …
1  Time (seconds), Requests, Throughput (requests…
2  Transaction Type Index, Transaction Name, Star…
3  Delivery, NewOrder, OrderStatus, Payment, Stoc…
4  kind, shape_key, sample_sql, executions, total…
5  Time (seconds), Throughput (requests/second), …
6  Time (seconds), Requests, Throughput (requests…
7  Transaction Type Index, Transaction Name, Star…
8  Delivery, NewOrder, OrderStatus, Payment, Stoc…
9  kind, shape_key, sample_sql, executions, total…
```

[4]:
```python
summary_rows = []
for engine, run in runs.items():
    parsed = parse_terminal_log(run['terminal_log'])
    summary = run['summary']
    latency = summary['Latency Distribution']

    summary_rows.append({
        'Engine':
        run['display_name'],
        'Run Label':
        run['label'],
        'Measured Requests':
        summary['Measured Requests'],
        'Elapsed Seconds':
        summary['Elapsed Time (nanoseconds)'] / 1_000_000_000,
        'Throughput (req/s)':
        summary['Throughput (requests/second)'],
        'Goodput (req/s)':
        summary['Goodput (requests/second)'],
        'Requests per Minute':
        summary['Throughput (requests/second)'] * 60,
        'Throughput - Goodput Gap':
        summary['Throughput (requests/second)'] -
        summary['Goodput (requests/second)'],
        'Average Latency (ms)':
        latency['Average Latency (microseconds)'] / 1000,
        'Median Latency (ms)':
        latency['Median Latency (microseconds)'] / 1000,
        'P95 Latency (ms)':
        latency['95th Percentile Latency (microseconds)'] / 1000,
        'P99 Latency (ms)':
        latency['99th Percentile Latency (microseconds)'] / 1000,
        'Max Latency (ms)':
        latency['Maximum Latency (microseconds)'] / 1000,
```

```
        'Terminal Throughput (req/s)':
        parsed.get('terminal_throughput'),
        'Terminal Goodput (req/s)':
        parsed.get('terminal_goodput'),
    })

summary_comparison_df = pd.DataFrame(summary_rows).set_index('Engine')
summary_comparison_df
```

[4]:

Engine	Run Label	Measured Requests	Elapsed Seconds	Throughput (req/s)	Goodput (req/s)	Requests per Minute	Throughput - Goodput Gap	Average Latency (ms)	Median Latency (ms)	P95 Latency (ms)	P99 Latency (ms)	Max Latency (ms)	Terminal Throughput (req/s)	Terminal Goodput (req/s)
dbfs	tpcc_2026-05-03_04-12-31	324	61.382	5.278	5.311	316.704	-0.033	184.257	8.366	1,457.553	1,770.172	3,246.423	5.278	5.311
PostgreSQL	tpcc_2026-05-03_23-26-55	24858	60.000	414.299	412.683	24,857.957	1.617	2.394	1.840	5.948	11.448	181.774	407.433	405.216

[5]:
```
transaction_names = [
    'NewOrder', 'Payment', 'OrderStatus', 'Delivery', 'StockLevel'
]
transaction_pattern = re.compile(r'procedures\.(\w+)/\d+\s+\[\s*(\d+)\]')


def extract_histogram_counts(log_text, section_name):
    section_pattern = re.compile(
```

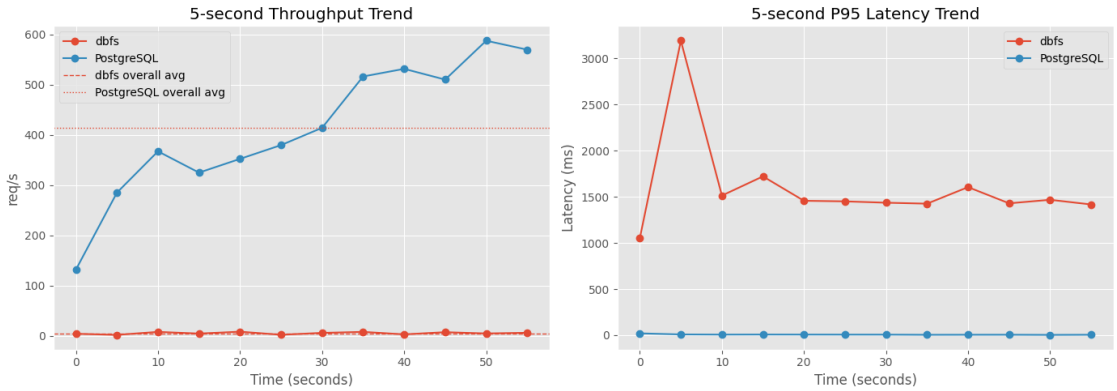
```
        rf'{section_name}:\n(.*?)(?:\n\w[\w ]+:|\Z)',
        re.DOTALL,
    )
    match = section_pattern.search(log_text)
    if not match:
        return {name: 0 for name in transaction_names}

    block = match.group(1).strip()
    if '<EMPTY>' in block:
        return {name: 0 for name in transaction_names}

    counts = {name: 0 for name in transaction_names}
    for name, count in transaction_pattern.findall(block):
        counts[name] = int(count)
    return counts


mix_rows = []
for engine, run in runs.items():
    completed = extract_histogram_counts(run['terminal_log'],
                                         'Completed Transactions')
    aborted = extract_histogram_counts(run['terminal_log'],
                                       'Aborted Transactions')
    completed_total = sum(completed.values())
    aborted_total = sum(aborted.values())

    for name in transaction_names:
        mix_rows.append({
            'Engine':
            run['display_name'],
            'Transaction':
            name,
            'Completed':
            completed[name],
            'Aborted':
            aborted[name],
            'Observed Mix (%)': (completed[name] / completed_total *
                                 100) if completed_total else 0.0,
            'Abort Rate (%)':
            (aborted[name] / (completed[name] + aborted[name]) * 100) if
            (completed[name] + aborted[name]) else 0.0,
        })

transaction_mix_df = pd.DataFrame(mix_rows)
transaction_mix_df
```

[5]:

	Engine	Transaction	Completed	Aborted	Observed Mix (%)	Abort Rate (%)
0	dbfs	NewOrder	145	0	44.479	0.000
1	dbfs	Payment	136	0	41.718	0.000
2	dbfs	OrderStatus	18	0	5.521	0.000
3	dbfs	Delivery	14	0	4.294	0.000
4	dbfs	StockLevel	13	0	3.988	0.000
5	PostgreSQL	NewOrder	11103	134	45.667	1.192
6	PostgreSQL	Payment	10298	0	42.356	0.000
7	PostgreSQL	OrderStatus	959	0	3.944	0.000
8	PostgreSQL	Delivery	1007	0	4.142	0.000
9	PostgreSQL	StockLevel	946	0	3.891	0.000

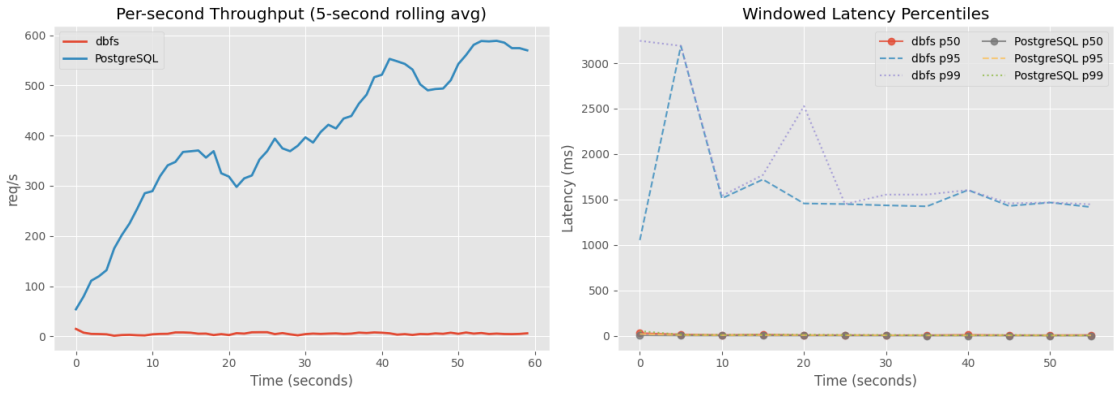
[6]:
```
fig, axes = plt.subplots(1, 2, figsize=(14, 5))

for engine, run in runs.items():
    results_df = run['results_df']
    label = run['display_name']
    axes[0].plot(
        results_df['Time (seconds)'],
        results_df['Throughput (requests/second)'],
        marker='o',
        label=label,
    )
    axes[1].plot(
        results_df['Time (seconds)'],
        results_df['95th Percentile Latency (millisecond)'],
        marker='o',
        label=label,
    )

axes[0].axhline(
    summary_comparison_df.loc['dbfs', 'Throughput (req/s)'],
    linestyle='--',
    linewidth=1,
    label='dbfs overall avg',
```

```
)
axes[0].axhline(
    summary_comparison_df.loc['PostgreSQL', 'Throughput (req/s)'],
    linestyle=':',
    linewidth=1,
    label='PostgreSQL overall avg',
)
axes[0].set_title('5-second Throughput Trend')
axes[0].set_xlabel('Time (seconds)')
axes[0].set_ylabel('req/s')
axes[0].legend()

axes[1].set_title('5-second P95 Latency Trend')
axes[1].set_xlabel('Time (seconds)')
axes[1].set_ylabel('Latency (ms)')
axes[1].legend()

fig.tight_layout()
plt.show()
```



[7]:
```
fig, axes = plt.subplots(1, 2, figsize=(14, 5))

for engine, run in runs.items():
    samples_df = run['samples_df'].copy()
    results_df = run['results_df']
    label = run['display_name']
    samples_df['Throughput Rolling'] = samples_df[
        'Throughput (requests/second)'].rolling(5, min_periods=1).mean()

    axes[0].plot(
        samples_df['Time (seconds)'],
        samples_df['Throughput Rolling'],
        linewidth=2,
```

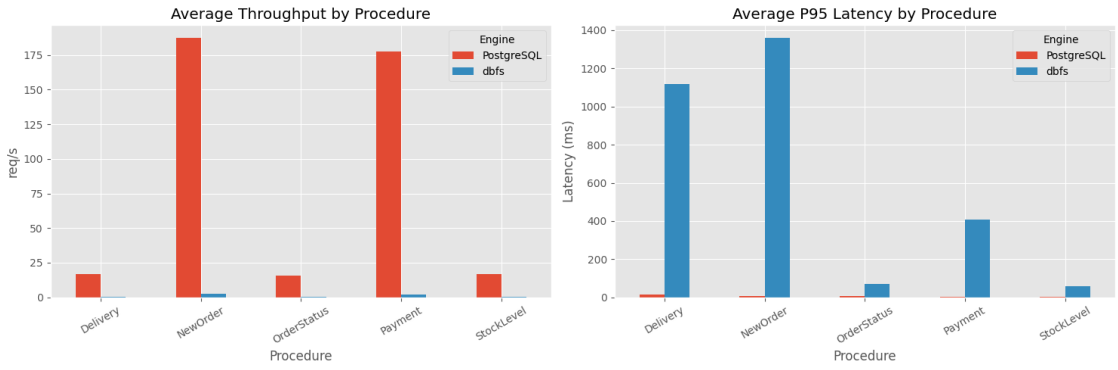
```
        label=label,
    )
    axes[1].plot(
        results_df['Time (seconds)'],
        results_df['Median Latency (millisecond)'],
        marker='o',
        alpha=0.8,
        label=f'{label} p50',
    )
    axes[1].plot(
        results_df['Time (seconds)'],
        results_df['95th Percentile Latency (millisecond)'],
        linestyle='--',
        alpha=0.8,
        label=f'{label} p95',
    )
    axes[1].plot(
        results_df['Time (seconds)'],
        results_df['99th Percentile Latency (millisecond)'],
        linestyle=':',
        alpha=0.8,
        label=f'{label} p99',
    )

axes[0].set_title('Per-second Throughput (5-second rolling avg)')
axes[0].set_xlabel('Time (seconds)')
axes[0].set_ylabel('req/s')
axes[0].legend()

axes[1].set_title('Windowed Latency Percentiles')
axes[1].set_xlabel('Time (seconds)')
axes[1].set_ylabel('Latency (ms)')
axes[1].legend(ncol=2)

fig.tight_layout()
plt.show()
```



[8]:
```
procedure_rows = []
for engine, run in runs.items():
    for procedure_name, procedure_df in run['procedure_dfs'].items():
        procedure_rows.append({
            'Engine':
            run['display_name'],
            'Procedure':
            procedure_name,
            'Avg Throughput (req/s)':
            procedure_df['Throughput (requests/second)'].mean(),
            'Avg Latency (ms)':
            procedure_df['Average Latency (millisecond)'].mean(),
            'Avg P95 Latency (ms)':
            procedure_df['95th Percentile Latency (millisecond)'].mean(),
        })

procedure_comparison_df = pd.DataFrame(procedure_rows).sort_values(
    ['Procedure', 'Engine'])
display(procedure_comparison_df)

procedure_throughput = procedure_comparison_df.pivot(
    index='Procedure', columns='Engine', values='Avg Throughput (req/s)')
procedure_p95 = procedure_comparison_df.pivot(index='Procedure',
                                              columns='Engine',
                                              values='Avg P95 Latency (ms)')

fig, axes = plt.subplots(1, 2, figsize=(15, 5))
procedure_throughput.plot(kind='bar', ax=axes[0])
axes[0].set_title('Average Throughput by Procedure')
axes[0].set_ylabel('req/s')
axes[0].tick_params(axis='x', rotation=30)

procedure_p95.plot(kind='bar', ax=axes[1])
```

```
axes[1].set_title('Average P95 Latency by Procedure')
axes[1].set_ylabel('Latency (ms)')
axes[1].tick_params(axis='x', rotation=30)

fig.tight_layout()
plt.show()
```

```
      Engine    Procedure  Avg Throughput (req/s)  Avg Latency (ms)  \
5  PostgreSQL     Delivery                  16.717             8.576
0       dbfs     Delivery                   0.233           988.930
6  PostgreSQL     NewOrder                 187.100             3.639
1       dbfs     NewOrder                   2.383           213.106
7  PostgreSQL  OrderStatus                  15.750             2.011
2       dbfs  OrderStatus                   0.300            65.629
8  PostgreSQL      Payment                 177.767             1.612
3       dbfs      Payment                   2.267            80.162
9  PostgreSQL   StockLevel                  16.967             1.232
4       dbfs   StockLevel                   0.217            57.830

   Avg P95 Latency (ms)
5                14.199
0             1,119.677
6                 6.476
1             1,358.201
7                 4.858
2                68.358
8                 3.111
3               409.163
9                 2.648
4                58.680
```



[9]:
```
mix_comparison_rows = []
for engine in transaction_mix_df['Engine'].unique():
    engine_mix = transaction_mix_df[transaction_mix_df['Engine'] ==
```

```
                                    engine].set_index('Transaction')
    for transaction_name, configured_ratio in configured_ratios.items():
        observed_ratio = engine_mix.loc[transaction_name, 'Observed Mix (%)']
        mix_comparison_rows.append({
            'Engine':
            engine,
            'Transaction':
            transaction_name,
            'Configured Ratio (%)':
            configured_ratio,
            'Observed Ratio (%)':
            observed_ratio,
            'Absolute Error (pp)':
            abs(observed_ratio - configured_ratio),
        })

mix_comparison_df = pd.DataFrame(mix_comparison_rows)
display(mix_comparison_df)

report_rows = []
for engine, run in runs.items():
    parsed = parse_terminal_log(run['terminal_log'])
    completed_total = transaction_mix_df.loc[transaction_mix_df['Engine'] ==
                                             run['display_name'],
                                             'Completed'].sum()
    aborted_total = transaction_mix_df.loc[
        transaction_mix_df['Engine'] == run['display_name'], 'Aborted'].sum()

    report_rows.append({
        'run_label':
        run['label'],
        'engine':
        run['display_name'],
        'dbms_type':
        parsed['dbms_type'],
        'workload':
        parsed['workload'],
        'scale_factor':
        float(parsed['scale_factor']),
        'terminals':
        int(parsed['terminals']),
        'duration_seconds':
        run['summary']['Elapsed Time (nanoseconds)'] / 1_000_000_000,
        'measured_requests':
        run['summary']['Measured Requests'],
        'throughput_req_per_s':
        run['summary']['Throughput (requests/second)'],
```

```
        'goodput_req_per_s':
        run['summary']['Goodput (requests/second)'],
        'completed_transactions':
        int(completed_total),
        'aborted_transactions':
        int(aborted_total),
        'abort_percentage':
        (aborted_total / (completed_total + aborted_total) * 100) if
        (completed_total + aborted_total) else 0.0,
    })

report_df = pd.DataFrame(report_rows)
display(report_df)
```

```
       Engine  Transaction  Configured Ratio (%)  Observed Ratio (%)  \
0        dbfs     NewOrder                45.000              44.479
1        dbfs      Payment                43.000              41.718
2        dbfs  OrderStatus                 4.000               5.521
3        dbfs     Delivery                 4.000               4.294
4        dbfs   StockLevel                 4.000               3.988
5  PostgreSQL     NewOrder                45.000              45.667
6  PostgreSQL      Payment                43.000              42.356
7  PostgreSQL  OrderStatus                 4.000               3.944
8  PostgreSQL     Delivery                 4.000               4.142
9  PostgreSQL   StockLevel                 4.000               3.891

   Absolute Error (pp)
0                0.521
1                1.282
2                1.521
3                0.294
4                0.012
5                0.667
6                0.644
7                0.056
8                0.142
9                0.109

                 run_label      engine dbms_type workload  scale_factor  \
0  tpcc_2026-05-03_04-12-31        dbfs  POSTGRES     TPCC         1.000
1  tpcc_2026-05-03_23-26-55  PostgreSQL  POSTGRES     TPCC         1.000

   terminals  duration_seconds  measured_requests  throughput_req_per_s  \
0          1            61.382                324                 5.278
1          1            60.000              24858               414.299

   goodput_req_per_s  completed_transactions  aborted_transactions  \
0              5.311                     326                     0
```

```
1            412.683                  24313                  134

   abort_percentage
0             0.000
1             0.548
```

[12]:
```python
def summarize_dbfs_query_trace(trace_df):
    summary_df = trace_df.copy()
    summary_df['sample_sql'] = summary_df['sample_sql'].fillna(
        summary_df['shape_key'])
    summary_df['total_elapsed_ms'] = summary_df['total_elapsed_us'] / 1000.0
    summary_df['mean_elapsed_ms'] = summary_df['mean_elapsed_us'] / 1000.0
    summary_df['p95_elapsed_ms'] = pd.NA
    summary_df['max_elapsed_ms'] = summary_df['max_elapsed_us'] / 1000.0
    summary_df = summary_df.sort_values(['total_elapsed_ms', 'executions'],
                                        ascending=[False, False])
    return summary_df[[
        'kind',
        'shape_key',
        'sample_sql',
        'executions',
        'total_elapsed_ms',
        'mean_elapsed_ms',
        'p95_elapsed_ms',
        'max_elapsed_ms',
        'failures',
    ]]


def summarize_postgres_server_query_trace(trace_df):
    summary_df = trace_df.copy()
    summary_df['kind'] = 'QUERY'
    summary_df['shape_key'] = summary_df['query'].str.replace(
        r'\s+', ' ', regex=True).str.strip()
    summary_df['sample_sql'] = summary_df['query']
    summary_df['executions'] = summary_df['calls']
    summary_df['total_elapsed_ms'] = summary_df['total_exec_time']
    summary_df['mean_elapsed_ms'] = summary_df['mean_exec_time']
    summary_df['p95_elapsed_ms'] = pd.NA
    summary_df['max_elapsed_ms'] = summary_df['max_exec_time']
    summary_df['failures'] = 0
    summary_df = summary_df.sort_values(['total_elapsed_ms', 'executions'],
                                        ascending=[False, False])
    return summary_df[[
        'kind',
        'shape_key',
        'sample_sql',
```

```
        'executions',
        'total_elapsed_ms',
        'mean_elapsed_ms',
        'p95_elapsed_ms',
        'max_elapsed_ms',
        'failures',
    ]]


def summarize_postgres_query_trace(trace_df):
    if {
            'query', 'calls', 'total_exec_time', 'mean_exec_time',
            'max_exec_time'
    }.issubset(trace_df.columns):
        return summarize_postgres_server_query_trace(trace_df)
    return summarize_dbfs_query_trace(trace_df)


def escape_plot_label(text):
    return text.replace('$', r'\$')


def build_shared_trace_comparison(trace_summaries):
    if not {'dbfs', 'postgres'}.issubset(trace_summaries):
        return None

    shared_df = trace_summaries['dbfs'].merge(
        trace_summaries['postgres'],
        on='shape_key',
        how='inner',
        suffixes=('_dbfs', '_postgres'),
    )
    if shared_df.empty:
        return shared_df

    shared_df['sample_sql'] = shared_df['sample_sql_dbfs'].fillna(
        shared_df['sample_sql_postgres'])
    shared_df['combined_total_elapsed_ms'] = (
        shared_df['total_elapsed_ms_dbfs'] +
        shared_df['total_elapsed_ms_postgres'])
    shared_df['combined_max_elapsed_ms'] = shared_df[[
        'max_elapsed_ms_dbfs',
        'max_elapsed_ms_postgres',
    ]].max(axis=1)
    shared_df['label'] = shared_df['sample_sql'].str.slice(0, 96)
    shared_df['plot_label'] = shared_df['label'].map(escape_plot_label)
    return shared_df.sort_values(
```

```
        ['combined_total_elapsed_ms', 'combined_max_elapsed_ms'],
        ascending=[False, False],
    )


trace_summaries = {}
for engine, run in runs.items():
    trace_df = run['query_trace_df']
    if trace_df is None or trace_df.empty:
        print(
            f"{run['display_name']}: {query_trace_filenames[engine]} was not␣
 ↪found."
        )
        continue

    if engine == 'dbfs':
        engine_summary_df = summarize_dbfs_query_trace(trace_df.copy())
    else:
        engine_summary_df = summarize_postgres_query_trace(trace_df.copy())

    engine_summary_df['Engine'] = run['display_name']
    engine_summary_df['time_share_pct'] = (
        engine_summary_df['total_elapsed_ms'] /
        engine_summary_df['total_elapsed_ms'].sum() * 100)
    engine_summary_df['label'] = engine_summary_df['sample_sql'].str.slice(
        0, 96)
    engine_summary_df['plot_label'] = engine_summary_df['label'].map(
        escape_plot_label)
    trace_summaries[engine] = engine_summary_df

for engine, summary_df in trace_summaries.items():
    display_name = runs[engine]['display_name']
    print(f'Top query shapes for {display_name}')
    display(summary_df[[
        'kind',
        'executions',
        'time_share_pct',
        'total_elapsed_ms',
        'mean_elapsed_ms',
        'p95_elapsed_ms',
        'max_elapsed_ms',
        'failures',
        'sample_sql',
    ]].head(15))

shared_trace_df = build_shared_trace_comparison(trace_summaries)
if shared_trace_df is not None:
```

```
overlap_summary_df = pd.DataFrame([{
    'dbfs_shapes':
    len(trace_summaries['dbfs']),
    'postgres_shapes':
    len(trace_summaries['postgres']),
    'shared_shapes':
    len(shared_trace_df),
    'dbfs_only':
    len(trace_summaries['dbfs']) - len(shared_trace_df),
    'postgres_only':
    len(trace_summaries['postgres']) - len(shared_trace_df),
}])
print('Shared query shape overlap')
display(overlap_summary_df)

display(shared_trace_df[[
    'kind_dbfs',
    'executions_dbfs',
    'executions_postgres',
    'total_elapsed_ms_dbfs',
    'total_elapsed_ms_postgres',
    'max_elapsed_ms_dbfs',
    'max_elapsed_ms_postgres',
    'sample_sql',
]].head(15))

fig, axes = plt.subplots(1, 2, figsize=(16, 8))

plot_specs = [
    (
        'combined_total_elapsed_ms',
        'total_elapsed_ms',
        'Shared Query Shapes by Total Time',
    ),
    (
        'combined_max_elapsed_ms',
        'max_elapsed_ms',
        'Shared Query Shapes by Max Latency',
    ),
]

for axis, (combined_metric, metric, title) in zip(axes, plot_specs):
    top_df = shared_trace_df.nlargest(
        10, combined_metric).sort_values(combined_metric)
    positions = list(range(len(top_df)))
    axis.barh(
        [position - 0.2 for position in positions],
```

```
            top_df[f'{metric}_dbfs'],
            height=0.4,
            label='dbfs',
            alpha=0.8,
        )
        axis.barh(
            [position + 0.2 for position in positions],
            top_df[f'{metric}_postgres'],
            height=0.4,
            label='PostgreSQL',
            alpha=0.8,
        )
        axis.set_yticks(positions)
        axis.set_yticklabels(top_df['plot_label'])
        axis.set_title(title)
        axis.set_xlabel('Latency (ms)')

    axes[0].legend()
    fig.tight_layout()
    plt.show()
```

```
Top query shapes for dbfs
```

	kind	executions	time_share_pct	total_elapsed_ms	mean_elapsed_ms	\
0	UPDATE	300040	49.986	59,029.550	0.196	
1	UPDATE	140	7.271	8,586.994	61.335	
2	QUERY	1358	6.305	7,445.932	5.483	
3	QUERY	140	5.658	6,681.869	47.727	
4	UPDATE	100000	4.218	4,981.063	0.049	
5	UPDATE	30000	3.982	4,702.779	0.156	
6	QUERY	1358	3.892	4,596.668	3.384	
7	UPDATE	1358	3.820	4,511.061	3.321	
8	UPDATE	100000	2.430	2,869.338	0.028	
9	QUERY	140	1.899	2,242.466	16.017	
10	UPDATE	136	1.392	1,643.903	12.087	
11	QUERY	136	1.266	1,495.317	10.994	
12	UPDATE	123	1.258	1,485.189	12.074	
13	UPDATE	145	1.200	1,416.818	9.771	
14	UPDATE	30000	1.024	1,209.098	0.040	

	p95_elapsed_ms	max_elapsed_ms	failures	\
0	<NA>	2,318.419	0	
1	<NA>	1,825.072	0	
2	<NA>	1,514.506	0	
3	<NA>	308.785	0	
4	<NA>	88.471	0	
5	<NA>	704.305	0	
6	<NA>	1,563.738	0	

```
7            <NA>       1,472.277         0
8            <NA>          64.824         0
9            <NA>       2,213.896         0
10           <NA>       1,623.694         0
11           <NA>       1,456.890         0
12           <NA>       1,461.621         0
13           <NA>       1,399.867         0
14           <NA>          55.695         0

                                          sample_sql
0   INSERT INTO order_line VALUES (1, 1, 1, 1, 200…
1   UPDATE order_line SET OL_DELIVERY_D = '2026-05…
2   SELECT S_QUANTITY, S_DATA, S_DIST_01, S_DIST_0…
3   SELECT SUM(OL_AMOUNT) AS OL_TOTAL FROM order_l…
4   INSERT INTO stock VALUES (1, 1, 37, 0.0, 0, 0,…
5   INSERT INTO customer VALUES (1, 1, 1, 0.460500…
6   SELECT I_PRICE, I_NAME , I_DATA FROM item WHER…
7   UPDATE stock SET S_QUANTITY = 54 , S_YTD = S_Y…
8   INSERT INTO item VALUES (1, 'sxvnjhpdqdxvc', 5…
9   SELECT O_C_ID FROM oorder WHERE O_ID = 2101 AN…
10  UPDATE warehouse SET W_YTD = W_YTD + 667.61999…
11  SELECT W_STREET_1, W_STREET_2, W_CITY, W_STATE…
12  UPDATE customer SET C_BALANCE = -677.619995117…
13  UPDATE district SET D_NEXT_O_ID = D_NEXT_O_ID …
14  INSERT INTO oorder VALUES (1, 1, 1, 2372, 5, 8…

Top query shapes for PostgreSQL

      kind  executions  time_share_pct  total_elapsed_ms  mean_elapsed_ms  \
0   UPDATE      300040          24.333        14,544.882            0.048
1    QUERY      111618          10.397         6,214.624            0.055
2    QUERY      111716           9.535         5,699.633            0.051
3   UPDATE      110712           7.052         4,215.096            0.038
4   UPDATE      100000           6.318         3,776.407            0.037
5    QUERY        6961           4.848         2,898.000            0.416
6   UPDATE      100000           3.097         1,851.217            0.018
7   UPDATE      110712           2.974         1,777.433            0.016
8   UPDATE       30000           1.721         1,028.426            0.034
9   UPDATE       11226           1.454           869.121            0.077
10  UPDATE       10040           1.442           861.959            0.085
11   QUERY        1018           1.427           853.245            0.838
12  UPDATE           1           1.422           849.863          849.863
13  UPDATE       30000           1.387           828.859            0.027
14  UPDATE       10666           1.385           827.680            0.077

   p95_elapsed_ms  max_elapsed_ms  failures  \
0            <NA>           0.625         0
1            <NA>          19.452         0
2            <NA>          16.140         0
```

```
3            <NA>           4.028         0
4            <NA>           0.364         0
5            <NA>          37.249         0
6            <NA>           0.320         0
7            <NA>           3.934         0
8            <NA>           0.177         0
9            <NA>          11.125         0
10           <NA>           5.951         0
11           <NA>          65.677         0
12           <NA>         849.863         0
13           <NA>           0.175         0
14           <NA>          11.730         0
```

```
                                        sample_sql
0   INSERT INTO order_line VALUES (1, 1, 1, 1, 953…
1   SELECT S_QUANTITY, S_DATA, S_DIST_01, S_DIST_0…
2   SELECT I_PRICE, I_NAME , I_DATA FROM item WHER…
3   INSERT INTO order_line (OL_O_ID, OL_D_ID, OL_W…
4   INSERT INTO stock VALUES (1, 1, 72, 0.0, 0, 0,…
5   SELECT C_FIRST, C_MIDDLE, C_ID, C_STREET_1, C_…
6   INSERT INTO item VALUES (1, 'sxvnjhpdqdxvcra',…
7   UPDATE stock SET S_QUANTITY = 42 , S_YTD = S_Y…
8   INSERT INTO customer VALUES (1, 1, 1, 0.048999…
9   INSERT INTO new_order (NO_O_ID, NO_D_ID, NO_W_…
10  UPDATE order_line SET OL_DELIVERY_D = '2026-05…
11  SELECT COUNT(DISTINCT (S_I_ID)) AS STOCK_COUNT…
12            DROP TABLE IF EXISTS order_line CASCADE
13  INSERT INTO oorder VALUES (1, 1, 1, 251, 4, 8,…
14  UPDATE warehouse SET W_YTD = W_YTD + 148.58999…
```

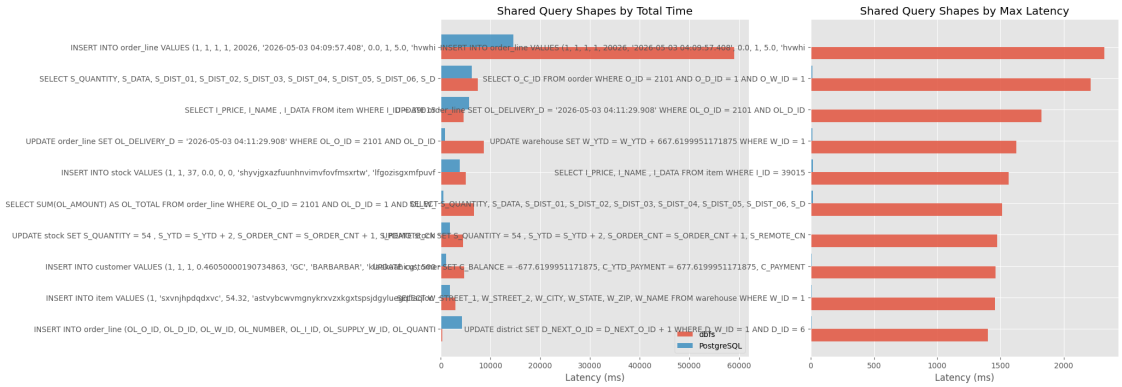
Shared query shape overlap

```
   dbfs_shapes  postgres_shapes  shared_shapes  dbfs_only  postgres_only
0           69               69             69          0              0
```

```
   kind_dbfs  executions_dbfs  executions_postgres  total_elapsed_ms_dbfs  \
0     UPDATE           300040               300040             59,029.550
2      QUERY             1358               111618              7,445.932
6      QUERY             1358               111716              4,596.668
1     UPDATE              140                10040              8,586.994
4     UPDATE           100000               100000              4,981.063
3      QUERY              140                10040              6,681.869
7     UPDATE             1358               110712              4,511.061
5     UPDATE            30000                30000              4,702.779
8     UPDATE           100000               100000              2,869.338
20    UPDATE             1358               110712                237.616
16     QUERY               85                 6961                930.027
9      QUERY              140                10040              2,242.466
10    UPDATE              136                10666              1,643.903
```

```
12    UPDATE             123                9592             1,485.189
14    UPDATE           30000               30000             1,209.098

    total_elapsed_ms_postgres  max_elapsed_ms_dbfs  max_elapsed_ms_postgres  \
0                   14,544.882           2,318.419                    0.625
2                    6,214.624           1,514.506                   19.452
6                    5,699.633           1,563.738                   16.140
1                      861.959           1,825.072                    5.951
4                    3,776.407              88.471                    0.364
3                      520.522             308.785                    6.876
7                    1,777.433           1,472.277                    3.934
5                    1,028.426             704.305                    0.177
8                    1,851.217              64.824                    0.320
20                   4,215.096               9.529                    4.028
16                   2,898.000              71.064                   37.249
9                      821.580           2,213.896                   11.735
10                     827.680           1,623.694                   11.730
12                     679.661           1,461.621                    8.251
14                     828.859              55.695                    0.175

                                        sample_sql
0   INSERT INTO order_line VALUES (1, 1, 1, 1, 200…
2   SELECT S_QUANTITY, S_DATA, S_DIST_01, S_DIST_0…
6   SELECT I_PRICE, I_NAME , I_DATA FROM item WHER…
1   UPDATE order_line SET OL_DELIVERY_D = '2026-05…
4   INSERT INTO stock VALUES (1, 1, 37, 0.0, 0, 0,…
3   SELECT SUM(OL_AMOUNT) AS OL_TOTAL FROM order_l…
7   UPDATE stock SET S_QUANTITY = 54 , S_YTD = S_Y…
5   INSERT INTO customer VALUES (1, 1, 1, 0.460500…
8   INSERT INTO item VALUES (1, 'sxvnjhpdqdxvc', 5…
20  INSERT INTO order_line (OL_O_ID, OL_D_ID, OL_W…
16  SELECT C_FIRST, C_MIDDLE, C_ID, C_STREET_1, C_…
9   SELECT O_C_ID FROM oorder WHERE O_ID = 2101 AN…
10  UPDATE warehouse SET W_YTD = W_YTD + 667.61999…
12  UPDATE customer SET C_BALANCE = -677.619995117…
14  INSERT INTO oorder VALUES (1, 1, 1, 2372, 5, 8…
```

```
/var/folders/fb/ffh7t7rs47jgclbwtxyfr75w0000gn/T/ipykernel_97268/379990782.py:20
0: UserWarning: Tight layout not applied. tight_layout cannot make Axes width
small enough to accommodate all Axes decorations
  fig.tight_layout()
```



1.1 Comparison Notes

Conclusion template:

- PostgreSQL shows a large throughput advantage over dbfs under the same TPCC configuration.
- dbfs completed the same workload shape without unexpected SQL errors, which means the current SQL support is sufficient for this TPCC slice.
- The main next step is performance-oriented investigation: loader behavior, per-procedure latency spikes, and the gap between dbfs and PostgreSQL throughput.
- If `dbfs_query_trace.csv` and `postgres_query_trace.csv` are present in the result directories, the notebook also ranks query shapes by total time, mean latency, and max latency to highlight bottlenecks on each engine.